

Autonomous Decentralised Search and Coverage with Unmanned Aerial Systems Swarms

By

Mahadev Reddy Devireddy Venkata

MSc Robotics Dissertation



School of Engineering Mathematics and Technology
UNIVERSITY OF BRISTOL

&

School of Engineering
UNIVERSITY OF THE WEST OF ENGLAND

A MSc dissertation submitted to the University of Bristol and the
University of the West of England in accordance with the
requirements of the degree of MASTER OF SCIENCE IN ROBOTICS in
the Faculty of Engineering.

Student ID: 2823511

Supervisor: Dr Davood Asadi

September 2026

Declaration of own work

I declare that the work in this MSc dissertation was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Research Degree Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, the work is the candidate's own work. Work done in collaboration with, or with the assistance of, others, is indicated as such. Any views expressed in the dissertation are those of the author.

Mahadev Reddy Devireddy Venkata, September 2026

Ethics statement

This project did not require ethical review as determined by my supervisor, Dr Davood Asadi. All work was simulation-based, with no human participants, animal subjects or physical hardware trials.

Mahadev Reddy Devireddy Venkata, September 2026

AI Usage Statement

Third-party AI tools were used in this project for debugging assistance with Python simulation code, for clarifying concepts in reinforcement learning and classical control, for the code and data audit reported in Section VI, and for structural and language feedback on written drafts. All experimental results were generated by code written for this project.

Acknowledgements

I thank Dr Davood Asadi for supervision throughout the project, and in particular for the feedback that prompted the extended evaluation in Section V and the diagnostic work in Section VI.

Supplementary material

Source code, all result files, the complete statistical output and the full figure set accompany this report as supplementary material.

Autonomous Decentralised Search and Coverage with Unmanned Aerial Systems Swarms

Reddy, Mahadev Devireddy Venkata

Abstract—UAV swarms operating without a central controller are attractive for search and coverage in disaster response, because losing any single agent does not end the mission. This work asks whether a decentralised multi-agent reinforcement learning policy, executing locally and passing no messages, can match classical reactive control on area coverage. Four controllers are compared on a 50×50 grid representing $150 \text{ m} \times 150 \text{ m}$ with six UAVs: Boids flocking, Artificial Potential Fields (APF), a single-agent PPO hybrid, and a parameter-shared multi-agent PPO policy in which all six UAVs learn and none is hand-coded. Obstacle density was varied from 0% to 30%, and every condition was run on 50 matched seeds across two disjoint seed sets, giving 1,600 runs. Each seed reproduces an identical obstacle layout for every controller, so comparisons use paired Wilcoxon signed-rank tests with Holm-Bonferroni correction, matched-pairs rank-biserial correlation and bootstrap confidence intervals. The decentralised MARL policy exceeds the 90% coverage target in all eight density-by-seed-set cells by mean, and significantly so in seven after correction, matching APF and beating the two other controllers, which never reach it. It beats APF significantly in open terrain (+4.7 and +5.0 percentage points at 0% obstacles) and is statistically indistinguishable from it at 10% and 20% on both seed sets, while APF retains an advantage at the hardest density on the development set (−3.8 points). All controllers transferred to held-out layouts within 2.4 points, none significantly. Auditing the apparatus produced three further findings. Changing only the reward scheme, from per-agent to the shared team reward the multi-agent literature warns against, costs 30.6 points, which exceeds the gap between any two controllers compared here. Repeating training at four seeds at 20% density shows the one-million-step budget is undertrained there: the across-seed spread is 7.8 points, closing to 4.2 at 1.85M steps where every seed overtakes APF, so at that density the budget and not the method sets the measured parity. And absolute-cell accounting shows that APF’s apparent improvement with obstacle density is a denominator artefact: it covers 640 fewer cells at 30% density than at 0%, while the free area shrinks faster. The same audit retracts a figure from an earlier draft of this work whose checkpoint could not be reproduced, and reverses the conclusion drawn from it.

I. INTRODUCTION

A. Motivation

Covering an unknown area with a team of unmanned aerial vehicles is a long-standing problem with immediate use in disaster response, wildfire monitoring and post-event damage assessment [1]. After an earthquake or flood, searching the affected ground quickly is what determines whether survivors are found. One UAV is too slow and too fragile. A centrally controlled team is faster, but the controller becomes a single point of failure, and in a collapsed or contested environment the communication link that centralisation depends on is exactly what fails first. Decentralised swarms avoid that by letting each agent decide from what it senses locally, so the

swarm keeps working when agents drop out [2].

B. Problem Statement

A decentralised swarm must cover as much of an unknown, obstructed area as possible within a fixed flight budget, using only local sensing and without exchanging messages. Two families of solution exist. Reactive controllers such as Boids and Artificial Potential Fields are cheap, need no training, and are the de facto baselines. Learned policies promise better coordination but face a scaling barrier: a joint action space over N agents with $|A|$ actions each grows as $|A|^N$, which is intractable beyond a handful of agents. Whether a learned decentralised policy can actually beat a well-tuned reactive controller on this task, under matched conditions and with evidence strong enough to believe, is unresolved.

C. Research Gap

Three gaps in the existing work motivated this project. Most studies of decentralised coverage test in empty or lightly obstructed environments [3], so it is unclear how methods behave as the environment hardens. Almost all report coverage percentage and stop, though battery use, collision risk and computational cost matter just as much once a system leaves simulation. And least discussed, the evidentiary standard is low: of the eight comparable studies in Section II, none reports a significance test, an effect size, or more than ten replications per condition. A learned method reported as beating a baseline by two points, from a single run, against a classical controller whose parameters are not stated, is not a result a reader can act on.

D. Research Questions

RQ1. Can a decentralised multi-agent PPO policy, in which every UAV learns from local observations and sends no messages, reach the 90% coverage criterion across obstacle densities from 0% to 30%?

RQ2. How does it compare against tuned Boids and APF controllers under matched obstacle layouts, and does the comparison survive correction for multiple testing?

RQ3. Do the controllers transfer to obstacle layouts unseen during development, and how do they compare on the secondary measures that determine deployability: energy, collision risk, computational cost, load balance and swarm-size scaling?

E. Contributions

- 1) A four-way comparison of Boids, APF, a single-agent PPO hybrid and a parameter-shared multi-agent PPO policy on identical matched obstacle layouts, with 50 replications per condition, paired significance

tests, effect sizes and bootstrap intervals. None of the comparable studies surveyed reports the last three.

- 2) A working decentralised MARL coverage system that matches or beats a tuned APF baseline, together with the two design decisions that make it learnable: a per-agent reward that repairs credit assignment, and a rollout length short enough to give PPO sufficient gradient updates.
- 3) A direct measurement of the credit-assignment problem the multi-agent literature describes. Holding network, budget, seed and dynamics fixed and changing only the reward scheme costs 30.6 percentage points of coverage, which is larger than the gap between any two controllers compared here.
- 4) Reporting of variance across training runs, which the surveyed literature omits, and evidence that both the variance and the parity with APF at intermediate density are artefacts of the training budget.
- 5) Evidence that percentage coverage over free cells is a misleading metric when free area varies with obstacle density, and absolute-cell accounting that corrects it.
- 6) A validation audit of the apparatus itself, with three repaired defects, one retracted figure and one withdrawn experiment documented in Section VI.

II. RELATED WORK

Bio-inspired flocking. Reynolds [4] introduced Boids as a distributed model in which each agent follows local rules computed from neighbours inside a perception radius, with no global state and no central controller. It is the origin of the claim that coherent group behaviour emerges from purely local rules, which is the claim decentralised reinforcement learning also makes. It was built for visual plausibility, not coverage: unmodified, its cohesion rule is harmful, since agents that stay together sweep the same cells, so any coverage use needs a dispersion term. Recent surveys [5], [6], [7] treat bio-inspired methods as the standard baseline for learned ones. Shang et al. [8] come closest to a bio-inspired coverage method, though their objective is exploration efficiency, not coverage at a fixed budget.

Artificial potential fields. Khatib [9] treats the robot as a particle under attraction to its goal and repulsion from obstacles. It is fully local, needs no map and costs constant time per step, which makes it the natural classical comparator for a learned local policy; Choset [10] places it in the wider coverage literature. Its known weakness is the local minimum in concave geometry. Which escape remedy is used strongly determines measured performance and papers routinely under-specify it, so an APF baseline with a stated escape rule and a published sweep is a harder comparator than the literature usually offers. Hickling et al. [11] show how the field now treats APF, using an APF-derived reward to train a TD3 policy rather than using APF as a controller. Du [12] gives a classical planning comparator using an enhanced A*.

MARL for coverage, and the credit-assignment problem. Schulman et al. [13] introduced PPO, now the default policy-gradient method. Fusco et al. [14] state the central difficulty

for the multi-agent case: the joint action space grows as $|A|^N$, which they call impractical for any significant number of agents, and full decentralisation scales but produces suboptimal actions and converges poorly under high partial observability. They report training time rising from 0.065 s at 16×10^3 cells to 3,354 s at 64×10^3 .

Two standard responses exist and this project used both in turn. The first is to sidestep the joint action space with hierarchy or assignment, which is what most empirical work does, and each such choice buys tractability by reintroducing something central. Yu et al. [15] use event-triggered hierarchical RL, where a high-level module assigns regions and low-level policies execute inside them; the assignment is a central decision, so the result speaks to hierarchical coordination rather than to what independent agents achieve. Leong et al. [16] learn selection over a discretised set of candidate viewpoints, which bounds the action space but presupposes a generator with global knowledge of the map. Xie et al. [17] decompose and allocate cells centrally, which is efficient and is not decentralised in the sense at issue here. None of the three establishes whether agents acting on local input alone can cover an unknown area, because in each the hardest coordination decision is taken somewhere the agents are not. The second response is *parameter sharing*, in which one policy is applied independently to every agent so the action space stays Discrete($|A|$) while all agents learn. It is the only one of the two that leaves the question intact, and it is the route taken here.

Parameter sharing alone is not sufficient, because a shared policy trained on a shared team reward suffers the credit-assignment problem: an agent’s reward is dominated by the behaviour of teammates it does not control, so the correlation between its own action and its own return is small and it learns little. The pathology is well documented under the name of the lazy agent, and the standard responses are difference rewards and counterfactual baselines, as in COMA [18], or value decomposition, as in VDN [19] and QMIX [20]. This literature is directly relevant, because Section VI-A shows the earlier hybrid failed for exactly this reason, and Section III-F adopts the simplest effective remedy: each agent is rewarded for the cells it is first to visit.

Evaluation practice, and why numerical comparison is not available. Replication counts across these papers are typically under ten where stated at all. None reports a hypothesis test between its method and its baselines, none reports an effect size, and train/test separation over layouts is rare. A like-for-like coverage comparison is therefore not possible. Fusco et al. report no coverage percentage at all: their environment terminates on full exploration, so coverage is 100% whenever an episode completes, and their metrics are energy, mission time and reward. The problem recurs because papers measure different quantities, use different termination rules so fixed-budget and time-to-completion framings are inverses, and differ in scale by more than an order of magnitude while coverage is strongly sensitive to the agent-to-area ratio. Table I therefore compares experimental design and evidentiary standard rather than headline numbers.

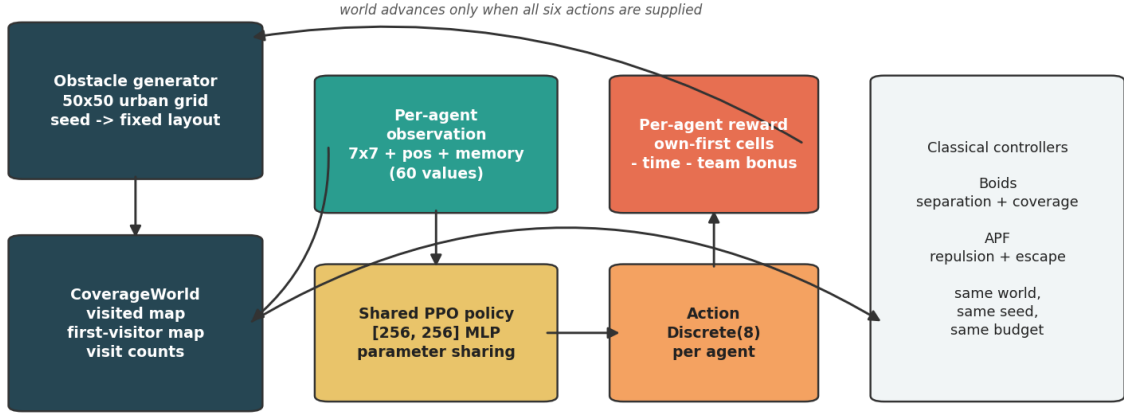


Fig. 1. System architecture. One world serves all four controllers on identical seeded layouts, advancing only once all six actions are supplied.

TABLE I

EXPERIMENTAL DESIGN COMPARED AGAINST RELATED WORK. N/S = NOT STATED.

Study	Coverage metric	Dens.	Seeds	Sig./ES
Reynolds [4]	none (animation)	no	—	no
Fusco [14]	none; energy, time	no	1	no
Yu [15]	3D region coverage	no	n/s	no
Leong [16]	viewpoint coverage	no	n/s	no
Xie [17]	reduction in grids	no	n/s	no
Shang [8]	exploration effc.	no	n/s	no
Hickling [11]	distance, makespan	no	n/s	no
Du [12]	path length	no	n/s	no
This work	% free cells and absolute cells	4	50	yes

III. METHODOLOGY

A. System Architecture

Fig. 1 shows how the pieces fit together. A single world holds the obstacle map, the shared visited map and the six UAV positions. It advances only when an action has been supplied for every agent, so all six move simultaneously within a timestep. The same world class serves all four controllers, and the same seed produces the same obstacle layout for each, which is what makes the comparison matched rather than merely parallel.

B. Environment Definition

A 50×50 discrete grid built in Python with Mesa [21]. At 3 m per cell the grid covers $150 \text{ m} \times 150 \text{ m}$, roughly a residential block. Six UAVs start spread around a shared HOME point at grid centre (25, 25), eight cells out on the four cardinals and $(\pm 5, \pm 5)$ on two diagonals, reflecting deployment from one launch site; a five-cell safe zone around HOME is kept clear of obstacles.

Early versions scattered single-cell obstacles at random, producing environments nothing like a real search area. The generator was rewritten to place rectangular building blocks three to eight cells on a side, with walls and L- and U-shaped structures, at four densities: 0%, 10%, 20% and 30% of grid area. Fig. 2 shows one such layout at 20%, with the swarm part-way through an episode.

Each episode runs 700 timesteps for every controller. Read against 3 m cells at one cell per step, that scales to roughly 3 m/s and a 12-minute mission, which sits inside the endurance of a typical multirotor. That scaling is an order-of-magnitude check and no more. The model is purely kinematic,

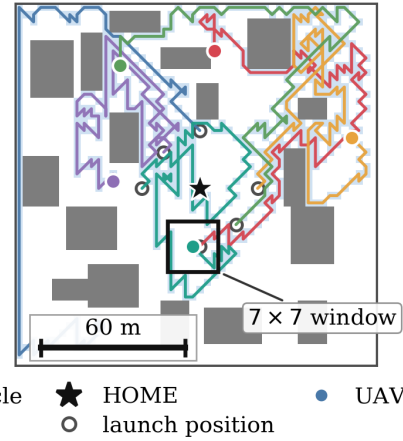


Fig. 2. The task. Six UAVs sweep a $150 \text{ m} \times 150 \text{ m}$ block at 20% density, at step 110 of 700 with 30% covered. Layout and paths are both real: the deployed policy on a layout the experiments ran. Paths zigzag because actions are sampled, not greedy (Section III-I). Each agent sees only a window like the one drawn, and nothing joins one agent to another.

with no inertia, turn radius, acceleration limit or wind, and diagonal moves cover $\sqrt{2}$ cells per timestep, so implied speed varies with heading. The figures show the episode length is not absurd; they do not support a flight-time claim. Coverage is the fraction of non-obstacle cells visited at least once,

$$C = \frac{|\mathcal{V}|}{|\mathcal{F}|} \times 100\% \quad (1)$$

with $\mathcal{V}$ the visited free cells and $\mathcal{F}$ all free cells. Section IV-D reports $|\mathcal{V}|$ directly alongside C , because the denominator changes with density.

C. Justification of the 90% Target

The criterion comes from search and rescue planning, where the governing quantity is probability of detection, a function of a coverage factor $C_f = n_s dW/A$ for n_s searchers travelling distance d with sweep width W over area A . Perkins [22] states that a probability of detection of 90% requires $C_f = 1.2$, and records that standard tactics reach 63%, 72% and 87%. None reaches 90%, so the value is a recognised and demanding planning target; the same framework underpins the US National Search and Rescue Supplement [23]. The construct differs from the one measured here, since Eq. 1

counts cell entry with a sensor that never fails and is therefore an upper bound on field performance. The criterion was fixed before any experiment ran and is not adjusted afterwards.

D. Observation Space

Each agent senses a 7×7 window centred on itself, a detection radius of three cells or 9 m, realistic for a depth sensor such as the Intel RealSense L515. The window encodes obstacles as -1.0 , unvisited free cells as 0.0 , visited cells as 1.0 and other agents as 2.0 , giving 49 values, plus two for normalised position (Fig. 3). Writing $p_i = (r_i, c_i)$ for agent i 's cell and $\delta \in \{-3..3\}^2$ for an offset within the window,

$$o_i[\delta] = \begin{cases} -1 & p_i + \delta \text{ obstacle or off-grid} \\ 2 & \text{occupied by another agent} \\ 1 & \text{visited by any agent} \\ 0 & \text{free and unvisited} \end{cases} \quad (2)$$

with $o_i \in \mathbb{R}^{60}$ after appending $(r_i/G, c_i/G)$ and the nine memory features of Section III-D.1. Cells outside the grid are also -1 , so an agent cannot distinguish the world boundary from an obstacle.

The visited channel is the swarm's shared record, not the agent's private one: a cell reads 1.0 once any agent has occupied it. An agent can tell from its window whether the ground ahead was already swept, but not by whom or when. This is indirect coordination through a shared trace, and on a real platform it would come from a shared occupancy grid or from visible evidence of a prior pass. No agent sends or receives a message, so the property under test is decentralised control without message passing; complete informational isolation is a stronger condition and this design does not meet it. The channel is identical for Boids and APF, so its advantages no one.

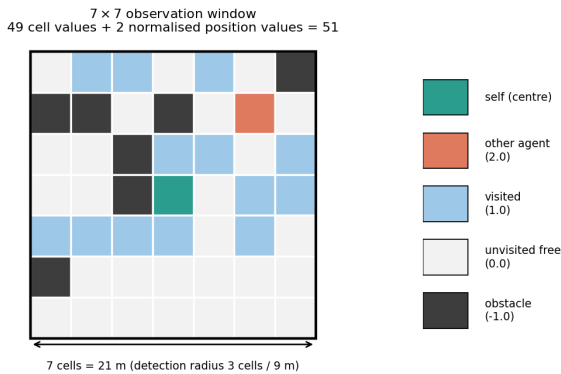


Fig. 3. The 7×7 observation window. Values are the same for every controller; only the learned policy additionally receives the nine memory features of Section III-D.1.

1) Onboard memory features

The sensor window gives no gradient towards unexplored ground once the immediate neighbourhood is swept, and a policy without one performs no better than a random walk. Each agent therefore maintains its own belief map of cells it has sensed, and derives nine features from it: four quadrant scores for remaining unexplored area, a normalised vector and distance to the nearest cell it believes unexplored, the

fraction of the arena it has sensed, and the fraction of the episode remaining. Two of the nine, the sensed fraction and the time remaining, are internal to the agent. The other seven are derived from a target set that combines what this agent has sensed with the same shared visited record the window already carries, so they extend its range but introduce no mechanism the window does not already use. They correspond to onboard map memory that any real UAV carries. The observation is 60 values in total. Section V-A measures what these features are worth, because they are information the classical controllers do not have, and the comparison against those controllers is therefore not one of equal information.

E. Action Space

Each UAV moves one cell per timestep in one of eight directions, a first-order kinematic model with no inertia, giving Discrete(8) per agent. Obstacle avoidance happens at the planning layer: cells containing obstacles are removed from the legal move set and each controller chooses among the remainder. Inertia, wind and battery drain beyond the mission budget are not modelled.

F. Reward Function and Gain Selection

Agent i receives

$$r_{i,t} = w_{cov} \Delta v_{i,t} - w_{time} + \mathbb{K}[\text{done}] w_{team} C_T / 100 \quad (3)$$

where $\Delta v_{i,t}$ counts the cells agent i was first to visit at step t , and C_T is final swarm coverage. Rewarding own-first-visits rather than the global new-cell count is what makes the signal a function of the agent's own action; Section VI-A shows what happens without it.

The three gains were set by scale reasoning rather than search, and the reasoning is worth stating because the relative magnitudes matter more than the absolute values (Fig. 4).

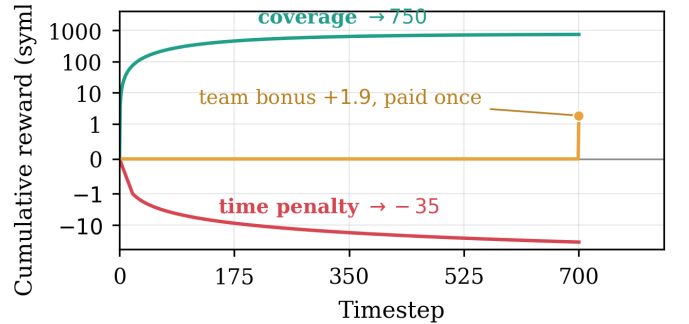


Fig. 4. What each reward term contributes to one agent over one episode, on a symmetric log scale. The coverage term dominates by more than an order of magnitude; the other two shape behaviour without competing with it.

$w_{cov} = 3.0$ and $w_{time} = 0.05$ set the exchange rate between finding new ground and spending time: one new cell is worth 60 steps of the time penalty. This must be large, because an agent that trades a new cell for a shorter path defeats the objective; but it must be finite, or a policy that stalls indefinitely near unexplored ground incurs no cost. Sixty steps is roughly a tenth of the episode, which makes loitering expensive without making the penalty compete with the coverage signal. Over a full episode the coverage term

accumulates to roughly 750 against -35 from the time penalty, so the ratio is deliberately lopsided.

$w_{team} = 2.0$ applies once, at termination, scaled by final swarm coverage. Its purpose is to retain a shared interest so that agents do not compete for the same cells, and its magnitude is set so that it cannot dominate: at most 2.0 against an accumulated ≈ 750 , about 0.3% of return. It is a tie-breaker between otherwise equivalent policies, not a driver. Setting it much larger reintroduces the credit-assignment problem the per-agent term exists to solve, since a large shared terminal payment is again mostly determined by teammates.

G. PPO Architecture

All six UAVs are driven by one shared policy acting on each agent’s own local observation (Fig. 5). No agent is hand-coded and no agent sends a message.

This is centralised training with decentralised execution. Training pools all six agents’ experience into one gradient update, which could not be done onboard. Execution is local: each agent runs its own copy of the network on its own 7×7 observation, nothing is transmitted and no controller is in the loop. A deployed swarm needs the shared network installed before flight and is independent after. Parameter sharing keeps the action space at Discrete(8) rather than $8^6 = 262,144$, which is the scaling wall Fusco et al. [14] describe and which a naive joint-action formulation hit directly during this project: the policy collapsed onto a single repeated action and never recovered.

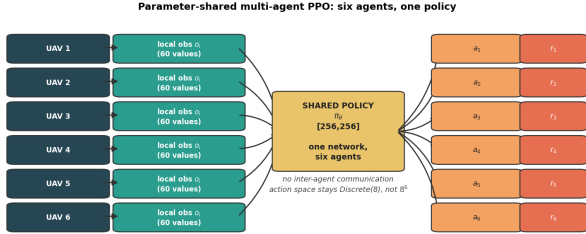


Fig. 5. Parameter-shared multi-agent PPO. One network produces an action for each agent from that agent’s own observation, and each agent’s experience is a separate sample for the same update.

The network is a $[256, 256]$ MLP with separate policy and value heads, the Stable-Baselines3 [24] `MlpPolicy` default. Six agents in each of eight parallel worlds present 48 agent-slots to the learner, so one policy update draws on all six agents’ experience simultaneously.

H. Training Procedure

Algorithm 1 states the loop. The centralised step is line 13, which pools all NW agent-slots into one gradient update; nothing else crosses between agents, and at execution only lines 8–9 run, on each agent’s own copy of π_θ .

Hyperparameters: $n_{steps} = 64$ per slot (3,072 samples per update), batch 512, four epochs, $\gamma = 0.99$, $\lambda = 0.95$, clip 0.2, entropy 0.005, learning rate 5×10^{-4} , one million agent-steps per density, $W = 8$ and $N = 6$. Training layouts are drawn from Seed Set A only.

Rollout length was the decisive hyperparameter. With 48 agent-slots, $n_{steps} = 256$ yields 12,288 samples per update but only about forty gradient updates per 500k steps, far too

Input: density d , budget T , seed pool S_A , W worlds, N agents
Output: one shared policy π_θ
1: initialise π_θ , a $[256, 256]$ MLP shared by all agents
2: $w_1..w_W \leftarrow$ worlds at density d , seeds from S_A
3: $t \leftarrow 0$
4: while $t < T$ do
5: $\mathcal{B} \leftarrow \emptyset$
6: for n_{steps} do
7: foreach world w , agent $i \in 1..N$ do
8: $o_i \leftarrow \text{OBSERVE}(w, i)$ // Eq. 2
9: $a_i \sim \pi_\theta(\cdot o_i)$ // sampled, not greedy
10: $\mathbf{r}, \text{done} \leftarrow \text{STEP}(w, \mathbf{a})$ // all N move together
11: $\mathcal{B} \leftarrow \mathcal{B} \cup \{(o_i, a_i, r_i)\}_{i=1}^N$ // r_i own-first, Eq. 3
12: if done then reset w with a fresh seed
13: $\theta \leftarrow \text{PPO-UPDATE}(\theta, \mathcal{B})$ // pools all NW slots
14: $t \leftarrow t + NW n_{steps}$
15: return π_θ

Algorithm 1: Parameter-shared MARL training (CTDE)

few for PPO to converge: coverage stalls near the random-walk level of 50% and does not improve with further training. Shortening the rollout to 64 gives roughly 160 updates over the same sample budget, and is what makes the task learnable. The effective number of updates falls as the slot count rises, which anyone scaling parameter sharing to many agents has to compensate for.

Training cost. On four CPU threads of an Apple M1 the implementation sustains about 11,400 agent-steps per second, so one million takes 1.5 minutes and the four-density set about six, with no GPU. Fusco et al. [14] report 3,354 s for one run at 64×10^3 cells; the environments differ too much for a direct comparison, but parameter sharing evidently keeps a decentralised policy within minutes of laptop compute at this scale. The classical controllers need no training at all, and that asymmetry belongs in any deployment decision.

I. Evaluation Procedure

Sampling at line 9 is a property of the deployed controller, not exploration noise left switched on. Six agents share one network, so a deterministic argmax makes every agent in an identical local situation choose the identical action, collapsing the swarm into one repeated move and costing almost ninety points of coverage when measured directly. Sampling breaks the symmetry between otherwise indistinguishable agents, and is the mechanism by which one shared network produces six behaviours. A fielded swarm would run the same stochastic policy. Sampling is seeded from the episode seed, so a seed reproduces the identical episode.

J. Classical-Controller Implementation

Boids [4] scores neighbouring cells as $s = w_{cov}c_{cov} + w_{sep}c_{sep} + \mathcal{U}(-0.1, 0.1)$, with c_{cov} equal to 2 for an unvisited cell and -1 for a visited one, and c_{sep} the normalised distance to the nearest agent. Agents move sequentially within a timestep, so each sees where the others have gone, which produces spatial separation without communication.

APF [9] scores a candidate cell as

$$s = c_{cov} - w_{rep} \sum_{o \in \mathcal{O}_n} \frac{k_{rep}}{d_o^2} + c_{sep} + \mathcal{U}(-0.1, 0.1) \quad (4)$$

over obstacles within radius r_{rep} , with c_{cov} of 5.0 for a cell unvisited by any agent, 1.0 for one absent from the agent’s own memory and -2.0 otherwise, and $w_{rep} = 0.3$. The

repulsion term of Eq. 4 is what Section V-D sweeps. An escape rule fires a random move to an unvisited neighbour after three stalled steps, which is what prevents the local-minimum collapse the method is known for. Rather than assert that $k_{rep} = 1.5$ and $r_{rep} = 2$ are well chosen, Section V-D reports the coverage surface over both.

K. Experimental Design and Statistical Methodology

The experiment crosses four controllers with four densities and two seed sets: 32 conditions, 50 seeds each, 1,600 runs. Seed Set A (0–49) was used for development and training, Seed Set B (50–99) reserved and used only at the end. Every controller sees the identical 50 layouts within a cell, as Algorithm 2 states: each passes its seed to the same obstacle generator, and the layouts are regenerated and compared cell by cell before any test runs, which is what licenses the paired design. The implementation is Python 3.13 with Mesa [21], Stable-Baselines3 [24] and Gymnasium; the bootstrap is seeded at 20260729, and code, results and checkpoints accompany this report.

Input: controllers $\mathcal{C}$, densities $\mathcal{D}$, seed sets $\mathcal{S}_A, \mathcal{S}_B$	
1: foreach $d \in \mathcal{D}$, $\mathcal{S} \in \{\mathcal{S}_A, \mathcal{S}_B\}$, seed $s \in \mathcal{S}$ do	
2: $M_s \leftarrow \text{GENOBSTACLES}(d, s)$; assert identical for every $c \in \mathcal{C}$	
3: $C[c, s] \leftarrow \text{RUN}(c, M_s, 700)$ for each $c \in \mathcal{C}$	
4: foreach pair (c_1, c_2) , each $(d, \mathcal{S})$ do	
5: Wilcoxon on $\{C[c_1, s] - C[c_2, s]\}_s$; r_{rb} , Cliff's δ , bootstrap CI	
6: Holm–Bonferroni within each comparison family	

Algorithm 2: Matched-layout evaluation

Three measures in Table IV each admit more than one reasonable definition, and every controller is measured on the same one. With u_i the distinct cells agent i occupied,

$$L = \ell \sum_{i=1}^N \sum_t \|p_{i,t} - p_{i,t-1}\|_2, \quad \mathcal{G} = \frac{2 \sum_k k u_{(k)}}{N \sum_k u_k} - \frac{N+1}{N} \quad (5)$$

with $\ell = 3$ m per cell, $u_{(k)}$ the k th smallest u_i , and $\mathcal{G} = 0$ perfectly even. A near-miss is a pair within one cell by Euclidean distance, counted before each move.

Twenty-seven of the 32 condition-level coverage distributions depart from normality under Shapiro-Wilk at $\alpha = 0.05$, so rank-based tests are primary. The design is matched, and this was verified rather than assumed: for each seed the obstacle map was regenerated independently through the Mesa generator and through both learned environments and compared cell by cell. The maps are identical for every sampled seed at every density, so observation i in each sample refers to the same environment and the paired Wilcoxon signed-rank test is correct. Mann-Whitney U [25] and Welch's t are reported as robustness checks. Holm-Bonferroni correction [26] is applied within each family. Effect sizes are matched-pairs rank-biserial correlation r_{rb} and Cliff's δ [27]; intervals are bootstrap percentile intervals from 10,000 resamples [28], and all error bars in this report are 95% confidence intervals on the mean rather than standard deviations.

IV. RESULTS

Table II reports final coverage for every controller at every obstacle density on both seed sets, with the number of runs

reaching the 90% target in parentheses.

TABLE II
FINAL COVERAGE (%), MEAN $\pm$ SD OVER 50 SEEDS, WITH RUNS
REACHING THE 90% TARGET IN PARENTHESES

Set	Controller	0%	10%	20%	30%
A	Boids	87.9 $\pm$ 4.5 (17)	86.7 $\pm$ 4.8 (12)	85.7 $\pm$ 5.6 (12)	84.6 $\pm$ 6.6 (9)
	APF	91.4 $\pm$ 3.6 (36)	92.5 $\pm$ 3.9 (39)	94.2 $\pm$ 3.5 (42)	94.7 $\pm$ 2.7 (47)
	Hybrid	85.8 $\pm$ 5.7 (13)	85.0 $\pm$ 5.5 (9)	84.8 $\pm$ 7.0 (13)	83.8 $\pm$ 6.1 (10)
	MARL	96.1 $\pm$ 1.5 (50)	93.7 $\pm$ 2.5 (45)	93.5 $\pm$ 3.7 (41)	90.8 $\pm$ 5.7 (35)
B	Boids	86.6 $\pm$ 4.6 (12)	86.8 $\pm$ 6.4 (18)	87.5 $\pm$ 4.9 (17)	83.6 $\pm$ 6.7 (7)
	APF	90.5 $\pm$ 4.1 (27)	93.0 $\pm$ 3.8 (41)	94.5 $\pm$ 2.8 (47)	94.0 $\pm$ 4.4 (42)
	Hybrid	85.6 $\pm$ 5.2 (11)	85.9 $\pm$ 4.7 (8)	84.4 $\pm$ 6.3 (9)	82.2 $\pm$ 6.8 (5)
	MARL	95.5 $\pm$ 1.8 (49)	93.8 $\pm$ 2.7 (46)	92.8 $\pm$ 3.8 (36)	93.1 $\pm$ 3.8 (41)

A. Meeting the 90% Criterion

Both the MARL policy and APF exceed 90% by mean in all eight of their cells. One-sided Wilcoxon tests against 90%, Holm-corrected over all 32 cells, confirm the excess is significant in seven cells each. The exceptions are the cells where the mean sits closest to the line: MARL at 30% density on the development set (90.84%, Holm $p = 0.499$) and APF at 0% density on the held-out set (90.46%, Holm $p = 1.000$). Boids falls significantly short in seven cells and is indistinguishable from 90% in one; the hybrid falls short in all eight. Only the two leading controllers meet the criterion at all, and neither dominates the other on it.

B. Pairwise Comparison

TABLE III
MARL AGAINST EACH BASELINE, PAIRED WILCOXON SIGNED-RANK,
HOLM-CORRECTED. DIFFERENCE IN PERCENTAGE POINTS.

Set	Dens.	Comparison	Diff.	95% CI	Holm p	r_{rb}
A	0%	MARL vs APF	+4.71	[3.67, 5.81]	<0.0001	+0.94
	10%	MARL vs APF	+1.21	[−0.09, 2.53]	0.5523	+0.24
	20%	MARL vs APF	−0.65	[−2.02, 0.72]	0.6892	−0.20
	30%	MARL vs APF	−3.82	[−5.59, −2.15]	0.0004	−0.64
	0%	MARL vs Boids	+8.19	[7.00, 9.40]	<0.0001	+1.00
	0%	MARL vs Hybrid	+10.28	[8.72, 11.95]	<0.0001	+1.00
B	0%	MARL vs APF	+5.02	[3.72, 6.31]	<0.0001	+0.87
	10%	MARL vs APF	+0.82	[−0.59, 2.30]	0.9649	+0.11
	20%	MARL vs APF	−1.73	[−2.99, −0.51]	0.2702	−0.33
	30%	MARL vs APF	−0.90	[−2.45, 0.68]	0.7116	−0.24
	20%	MARL vs Boids	+5.33	[3.69, 7.06]	<0.0001	+0.77
	20%	MARL vs Hybrid	+8.41	[6.41, 10.49]	<0.0001	+0.90

The full 48-comparison table, including every MARL-vs-Boids and MARL-vs-Hybrid cell and the Mann-Whitney and Welch robustness checks, is in the supplementary output.

Three findings follow. MARL beats APF in open terrain by 4.71 and 5.02 points, both highly significant with near-maximal effect sizes. The two cannot be separated at intermediate density: at 10% and 20% no comparison survives Holm correction on either seed set. APF keeps an edge at the hardest density by 3.82 points with a medium effect, but only on the development set; the same comparison on the held-out set is 0.90 points and not significant, so the advantage is not robust across layout samples. Against the two weaker baselines MARL wins everywhere by 5.3 to 10.9 points, r_{rb} 0.75 to 1.00, all sixteen surviving correction.

C. Generalisation

Seed Set B consists of layouts distinct from Set A, verified by regenerating and comparing maps. Changes from A to B for the MARL policy are +0.60, −0.11, +0.76 and −2.31

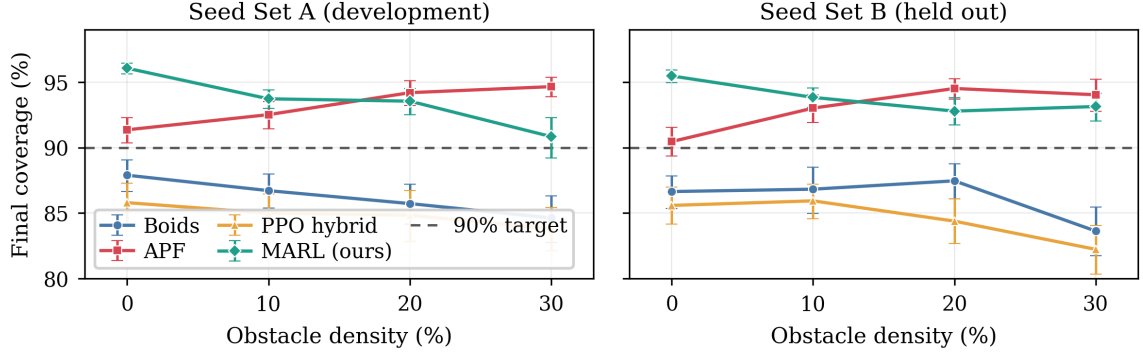


Fig. 6. Coverage against obstacle density for all four controllers, on the development and held-out seed sets. Error bars are 95% bootstrap confidence intervals on the mean. Only MARL and APF clear the target, crossing between 10% and 20% on both sets.

percentage points at 0–30% density; no controller changes by more than 2.31 points and no difference survives Holm correction. The policy was trained only on Set A layouts, so this is a genuine held-out test and shows no overfitting to the training maps; at 30% density it performs *better* on the unseen set. At 0% density there are no obstacles, so the two sets are the same environment and any difference there reflects only the stochastic policy.

D. Absolute Cells Against Percentage

Absolute-cell accounting resolves an ambiguity the earlier iteration of this project could not settle. APF’s percentage coverage rises by 3.3 points from 0% to 30% obstacles, which invites the reading that structured obstacles help it search. The absolute counts, recovered over the same 50 seeds by multiplying each run’s coverage by the free area of its layout, contradict that. APF covers 640 fewer cells at 30% than at 0%, a fall of 28%; Boids 727 fewer, 33%; and MARL 824 fewer, 34%. Every controller covers less ground as density rises, while the free area itself shrinks from 2,500 to 1,737 cells, a fall of 31%. APF’s percentage rises only because its absolute coverage falls more slowly than the denominator. No controller becomes a better searcher in cluttered terrain; APF degrades most gracefully. Percentage coverage alone is a misleading metric across varying free area, and reporting both is the correct practice.

Fig. 7 shows what that difference looks like on the ground: three controllers sweeping one identical 20% layout, where APF’s corridor-following and MARL’s outward partition are visible as search patterns rather than as summary statistics.

V. EXTENDED ANALYSIS

Coverage alone is a poor guide to field viability. Following supervisory feedback, further analyses were run at the same 700-step budget on 50 matched seeds, except the swarm-size and hybrid sweeps, which need a run per configuration and use 15.

Load balance. APF spreads work most evenly, MARL second, Boids third. The hybrid is far worse, and the per-agent breakdown explains why: at 20% obstacles its agents contribute 0.4%, 19.6%, 19.7%, 20.9%, 19.6% and 19.8% of unique coverage. Agent 0, the only one under policy control, accounts for 0.4% against a balanced 16.7%. MARL shows no such pathology and reaches near-APF balance with no

TABLE IV

SECONDARY MEASURES, MEAN OVER 50 SEEDS, DEFINED AT EQ. 5. NEAR-MISSES ARE A PROXIMITY PROXY, NOT OBSERVED COLLISIONS. [†]PER-STEP COST RE-MEASURED FOR ALL FOUR ON ONE MACHINE; ONLY THE ORDERING IS MEANINGFUL.

Measure	Controller	0%	10%	20%	30%
Path length	Boids	14913	14860	14819	14747
	APF	14946	14837	14723	14582
	Hybrid	13927	12744	12690	13505
	MARL	15240	14399	13696	11950
Gini	Boids	0.067	0.077	0.087	0.103
	APF	0.029	0.039	0.050	0.083
	Hybrid	0.197	0.228	0.239	0.244
	MARL	0.058	0.051	0.064	0.103
Near-misses	Boids	1244.4	1088.4	888.9	701.5
	APF	1245.4	1109.4	938.7	606.6
	Hybrid	0.8	0.9	0.7	0.8
	MARL	47.8	55.6	50.1	69.6
ms / step [†]	Boids	0.085	0.212	0.327	0.431
	APF	0.358	0.581	0.652	0.776
	Hybrid	0.412	0.418	0.408	0.392
	MARL	0.728	0.889	0.702	0.683

agent hand-coded to do so, because each is rewarded for its own contribution.

Proximity events. A near-miss proxies collision risk and counts no collisions: the grid gives a vehicle no body, so the measure captures how often a real airframe would have had to give way. MARL records roughly twenty times fewer than either classical controller while covering more ground. The hybrid’s near-zero figure comes from elsewhere: read with the load-balance result, it describes its single learned agent staying out of the way.

Computational cost. Boids and APF are $O(N^2)$ per timestep, each agent checking every other; a shared policy is $O(N)$, one forward pass each. The constants reflect that: the classical pair rises with density, the learned ones stay flat, and at six agents MARL is still dearest of the four, a forward pass costing more than scanning five neighbours. The scaling favours it; at this size it buys coverage with compute.

Swarm size. All four controllers were run at 3, 6, 9 and 12 agents at 20% density over 15 seeds (Table V). MARL and the hybrid were trained at six agents, so other sizes are a transfer test; the shared-policy architecture makes this possible at all, since one network applies to any swarm size without retraining.

Two things stand out. The MARL policy leads at three agents by 4.7 points over APF and by 10.1 over Boids, which

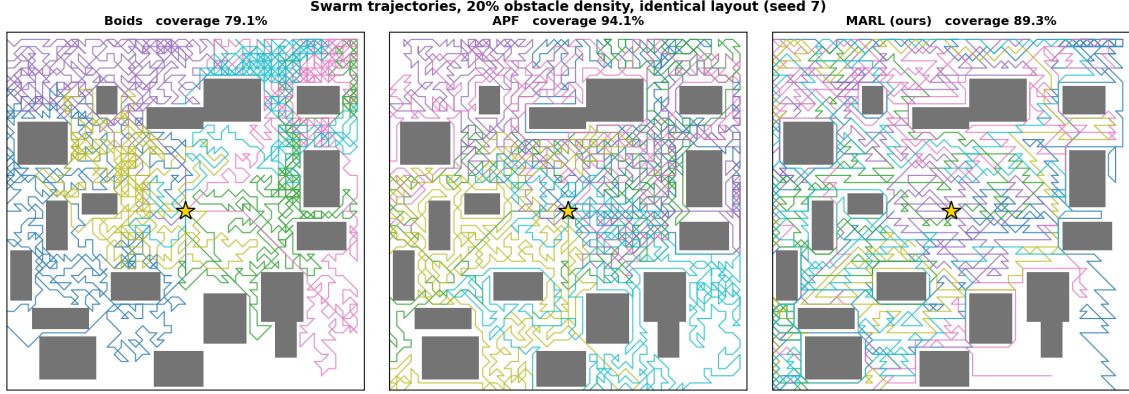


Fig. 7. Swarm trajectories on one identical layout at 20% obstacle density. Boids doubles back in open areas; APF follows the corridors between buildings; the MARL policy sweeps outward and partitions the arena without communicating. $\star$ marks HOME. Equivalents at every density are in the supplementary material.

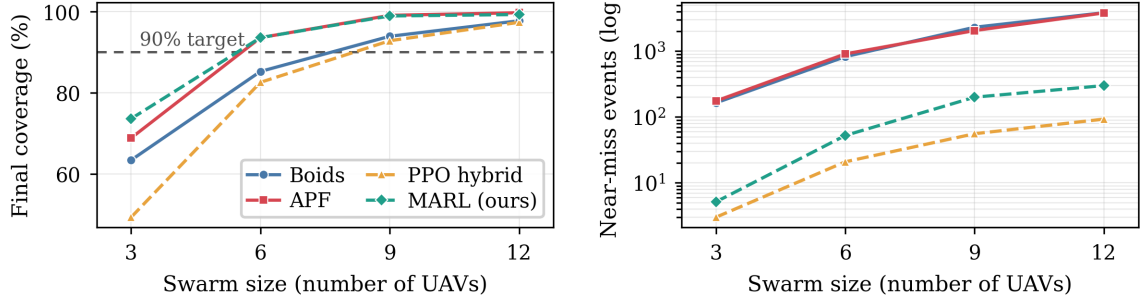


Fig. 8. Coverage and near-misses against swarm size at 20% density, 15 seeds, the same runs as Table V. The learned controllers are dashed: both were trained at six agents and tested outside that configuration at every other size. Log axis on the right, where they stay about an order of magnitude below the classical pair throughout.

TABLE V

COVERAGE AND NEAR-MISSES BY SWARM SIZE, 20% OBSTACLES, 15 SEEDS. MARL AND THE HYBRID ARE SIX-AGENT POLICIES TESTED AT OTHER SIZES. ALL FOUR COLUMNS COUNT A NEAR-MISS IDENTICALLY: EUCLIDEAN RADIUS ONE CELL, SAMPLED BEFORE EACH STEP.

N	Coverage (%)				Near-misses			
	Boids	APF	Hyb.	MARL	Boids	APF	Hyb.	MARL
3	63.5	68.9	49.4	73.6	185	175	2.2	5
6	85.2	93.5	82.6	93.6	827	910	20.9	52
9	93.9	99.0	92.8	98.9	2312	2059	55.7	200
12	97.7	99.7	97.3	99.2	3845	3819	92.9	300

is the condition where coordination matters most because there is least redundancy to hide a poor sweep. It converges with APF at six and nine and falls slightly behind at twelve, where the arena is saturated and any reasonable controller approaches the ceiling. Returns fall off past nine agents for every controller while near-misses rise four- to sixfold from six to twelve (Fig. 8), so six agents sits on the steep part of the curve where coordination differences are amplified. Across all sizes the learned controllers record twelve to thirty-four times fewer near-misses than the classical ones.

Moving obstacles. With obstacles relocating every 50 steps at 20% density, APF falls from 94.0% to 85.9% as the moving share goes from none to half, while Boids drops only from 85.9% to 82.8%. APF needs a field consistent long enough to guide an agent through; Boids recomputes each step. The learned policies were not retrained under this perturbation, so this covers the classical pair only.

A. Information Asymmetry: the Memory Ablation

The learned controller carries nine onboard-memory features (Section III-D.1) that Boids and APF do not have. Rather than declare this as a caveat, it was measured. An otherwise identical policy was trained with those features zeroed, leaving it exactly the information the classical controllers receive: a 7×7 sensed window and its own position.

TABLE VI

MEMORY-FEATURE ABLATION, 20% DENSITY, 50 MATCHED SEEDS. THE ABLATED POLICY KEEPS THE SAME 60-DIMENSIONAL VECTOR WITH THE NINE MEMORY FEATURES HELD AT ZERO

Configuration	Steps	Coverage (%)	$\geq 90\%$
MARL, memory	1.00M	93.6 ± 3.7	41/50
MARL, memory	1.85M	98.8 ± 1.0	50/50
MARL, no memory	1.00M	89.6 ± 4.6	31/50
MARL, no memory	1.85M	90.0 ± 4.2	29/50
APF (same information)	—	94.2 ± 3.5	42/50

At a matched budget of one million agent-steps the memory features are worth +4.00 percentage points (paired Wilcoxon, $p < 0.0001$, $r_{rb} = 0.72$), and without them the policy reaches 89.6% against APF's 94.2%. On equal information and equal training the classical controller wins, and it still wins at 1.85M (-4.22 pp, $p < 0.0001$).

Running both arms on to 1.85M separates two explanations that the single-budget comparison cannot. The memory-free policy gains nothing from the extra 850k steps (+0.43 pp, $p = 0.83$): it has converged. The memory policy gains +5.26 pp over the same interval and reaches 98.8%, so the gap

widens from 4.00 to 8.83 points ($r_{rb} = 1.00$). Extra training does not close it, which rules out the head-start explanation. What the features change is what the architecture can represent from local sensing alone: a 7×7 window and a position alone do not reach the target reliably.

B. Credit Assignment Measured Directly

Section III-F rewards each agent for the cells it is first to visit. The alternative that the literature warns against is the naive shared reward, in which every agent receives the global new-cell count. Difference rewards [18] and value decomposition [19], [20] exist to repair that arrangement, and it can be run here as a controlled experiment, since the network, budget, seed, observation and dynamics all stay fixed and only the reward scheme moves.

The shared-reward policy reaches $63.0 \pm 6.3\%$ against $93.6 \pm 3.7\%$ for the per-agent reward, a gap of 30.60 percentage points ($p < 10^{-14}$, $r_{rb} = 1.00$), and not one of its fifty runs reaches the target. On this task credit assignment decides whether the controller works at all: on the shared reward it loses to every baseline. It also accounts for the hybrid in Section VI-A without appeal to its architecture, since the hybrid used exactly this reward.

C. Training-Seed Variance and Budget

Every MARL figure reported above comes from one training run at PPO seed 0. Fifty evaluation seeds control layout variance; they say nothing about variance across training runs. Three further policies were trained at seeds 1, 2 and 3 under an identical budget and evaluated on the same fifty layouts (Table VII).

TABLE VII

COVERAGE BY PPO TRAINING SEED, 20% DENSITY, 50 MATCHED EVALUATION SEEDS EACH. APF SCORES 94.2% ON THE SAME LAYOUTS. SD_{SEED} IS ACROSS THE FOUR MEANS, NOT ACROSS RUNS

Budget	s0	s1	s2	s3	Mean	SD_{seed}	$\geq 90\%$
1.00M	93.55	92.98	91.08	85.75	90.84	3.55	122/200
1.85M	98.81	98.87	96.81	94.69	97.30	1.98	193/200

At 1.00M the spread is wide: 7.8 points between the best and worst seed, and seed 0 is the best of the four. The headline MARL numbers therefore sit at the top of their distribution, not in the middle of it. A single fresh training run at 20% density averages 90.8%, with a standard deviation of 5.0 points across the 200 individual runs, and that puts it behind APF.

The 1.85M row identifies the cause. The across-seed standard deviation falls from 3.55 to 1.98 points, the worst seed rises to 94.7% and overtakes APF, and every seed clears the target on at least 44 of 50 layouts. Both the shortfall and the instability at 1.00M come from stopping on the steep part of the learning curve. At 1.85M the same policy beats APF by 4.61 points at 20% density ($p < 10^{-14}$, $r_{rb} = 1.00$), so what Table III records at that density is the training budget talking and not a ceiling on the method. Holding the budget fixed across every controller and density is the right control for the main comparison, but at 20% it understates the learned controller.

How far that generalises is untested. The extended-budget evidence comes from one density, and 10% is a different regime with a smaller starting margin and sparser structure for APF to exploit. The same direction is plausible there, but plausible is an inference, not a measurement. Repeating the protocol at every density is future work.

D. APF Parameter Sensitivity

The repulsion parameters were swept over $k_{rep} \in \{0.5, 1.0, 1.5, 2.5, 4.0\}$ and $r_{rep} \in \{1, 2, 3, 4\}$ at every density. The reported configuration (1.5, 2) sits on a broad plateau: coverage varies by under two points across the centre of the grid and degrades sharply only at $r_{rep} = 4$, where the repulsion radius approaches the corridor width. APF's showing is a property of the method here, not of a fortunate parameter choice, which is what makes it a fair comparator. The full surface is in the supplementary output.

VI. VALIDATION AND THREATS TO VALIDITY

Three defects found by auditing the experiment code were repaired before any result above was generated, and one earlier experiment is withdrawn. A fourth item, the ablation in Section VI-A, is reported as a finding rather than a defect.

A. The Hybrid Could Not Answer the Question

An earlier iteration used one policy-controlled agent alongside five hand-written greedy agents, an architecture common enough to be worth reporting on. It carried the shared-reward defect quantified in Section V-B, and a second one that is fatal on its own. Replacing the learned agent with an inert, a uniformly random, a greedy and the trained agent gives swarm coverage of 82.21%, 82.37%, 84.07% and 84.83% at 20% density over 15 seeds. The trained policy is the best of the four, so it had learned something, but the entire range spanned by completely different behaviours for that one agent is 1.86 percentage points against a 9.37-point gap to APF. No agent could have closed it. The hybrid's deficit is a property of its five reactive agents, not of the learned one, and no conclusion about PPO follows from it.

The fully decentralised architecture in Section III-G removes both problems, and the 10.25-point margin between MARL and the hybrid in Table III quantifies what the shortcut cost.

B. Repaired Defects

Uncontrolled obstacle seeding. The Mesa models originally drew the obstacle seed from Mesa's internal generator, not the seed supplied by the runner, so one seed gave different layouts on repeat: 503, 510 and 512 obstacle cells across three runs. Both models now take an explicit seed, verified cell by cell, which is what licenses the paired tests.

Seed Set B was not held out. The earlier PPO environment mapped the episode seed with $s = \text{ep_seed} \% 50$, so Set B resolved to Set A's fifty layouts. The modulo was removed and the sets are now disjoint. **Unequal episode budgets.** Boids ran for 800 steps against 700 for the others; all four now use 700.

A retracted ablation figure. An earlier draft reported the memory-free policy at 95.0% after 1.85M steps and concluded

the memory features bought convergence speed alone. The surviving checkpoint re-evaluates to the 1.00M figure exactly, 89.55%, and no checkpoint matching 95.0% exists. Retrained from 1.00M to 1.85M it scores $90.0 \pm 4.2\%$, which is the value in Table VI, and the conclusion reverses.

C. Withdrawn and Outstanding

The sensor-noise experiment is withdrawn. For Boids and APF the perturbation corrupted the obstacle grid itself, once per episode, marking roughly 398 of 1,991 free cells unreachable and imposing a mechanical ceiling near 80% before the controller acted at all. For the hybrid it entered only the observation of the single learned agent, which contributes 0.4% of coverage. The comparison is between two different interventions and supports no conclusion. A valid version requires the same corruption, representation and persistence for every controller.

Redundancy is recorded for the MARL controller but not retrospectively for the classical ones, so no cross-controller comparison is made.

VII. DISCUSSION

Answering the question. A decentralised multi-agent PPO policy, in which every UAV learns from a local window, executes on its own network and exchanges no messages, reaches 90% coverage at every obstacle density tested and on layouts it has never seen. APF does so too, on the same layouts and by the same test; the two learned-free baselines do not. Against a well-tuned APF baseline it wins clearly in open terrain, ties at intermediate density, and at the hardest density trails on the development set but not on the held-out one. The answer to the research question is therefore yes for the criterion and qualified for the comparison: decentralised learning matches classical reactive control on this task, and beats it where the geometry offers least structure.

Why the crossover occurs. The two controllers fail in opposite regimes, and the crossover in Fig. 6 is the most informative result here. In an empty grid APF has little to work with: six agents under mutual repulsion spread evenly and then have nothing to distinguish one direction from another, so they re-sweep ground. The learned policy carries a directional signal from its own belief map and keeps moving towards unexplored space, which is exactly what an empty arena rewards. As obstacles increase, the situation narrows. Walls and corridors give APF's repulsion term the structure it needs, channels agents along streets and decorrelates their paths, while the learned policy faces a harder navigation problem with a 7×7 window that cannot see around a building. That the 30% result is significant on one seed set and not the other suggests the remaining gap is small relative to layout-to-layout variation. The classical controller degrades most gracefully in clutter; the learned one exploits open space best.

Section V-C limits how far that reading extends. The crossover is measured at a fixed budget, and at 1.85M the learned controller reaches 98.8% at 20% density and overtakes APF there on every training seed. What the main experiment establishes is that the two are close under equal training, not that clutter imposes a ceiling the learned controller cannot

pass.

What the absolute-cell result changes. No controller improves in absolute terms as obstacles increase, contrary to what percentage coverage suggests. This matters beyond the present study: coverage over free cells is the standard metric across the literature surveyed in Section II, so any paper varying obstacle density while reporting only that ratio invites the same misreading.

Relation to the literature. Fusco et al. [14] predict that fully decentralised MARL underperforms under high partial observability, and that the joint action space makes the design intractable at scale. This project confirms the second half directly, having hit the 8^6 wall, and qualifies the first: with parameter sharing, a per-agent reward and onboard memory, a fully decentralised policy is competitive with, and in open terrain better than, a tuned classical controller. The result suggests the pessimism is about the naive formulation rather than about decentralisation itself. The credit-assignment literature [18], [19], [20] is the reason, and Section V-B puts a number on it: the same network on the shared team reward those papers set out to repair loses 30.6 points and never reaches the target. The difference between a policy that fails and one that beats APF was, in this project, mostly a matter of what the agent was rewarded for.

Practical implication. For open or lightly cluttered search areas the learned controller covers most ground, and it records between nine and twenty-six times fewer proximity events than either classical controller, the margin narrowing as density rises. For dense urban clutter a tuned potential field clears the same target, remains slightly ahead on coverage and is far cheaper to deploy, needing no training and no GPU; the learned controller is worth its cost only if trained past the point where its across-seed spread closes. And for any mission, adding agents rivals changing algorithm: six to nine UAVs gains 5.3 to 10.2 points, against an 11.0-point spread between the best and worst controller at six.

VIII. CONCLUSIONS AND FUTURE WORK

A parameter-shared multi-agent PPO policy driving six UAVs from local observations, with decentralised execution and no message passing, achieved 90.8% to 96.1% coverage across obstacle densities from 0% to 30% on both development and held-out layouts. Its mean exceeds the 90% search-and-rescue planning target in every condition tested, and does so significantly in seven of eight after correction; APF matches that record and the other two controllers never reach the target. It beat a tuned APF controller by 4.7 and 5.0 percentage points in open terrain and was statistically indistinguishable at 10% and 20% density; at 30% it trailed by 3.8 points on the development set, a difference that did not replicate on the held-out set. It beat a Boids baseline and a single-agent hybrid by 5.3 to 10.9 points throughout. Transfer to fifty unseen layouts changed coverage by at most 2.31 points and was never significant.

Two design decisions account for that performance, and the ablations size both. Replacing the per-agent reward with the shared team reward, and changing nothing else, costs 30.6

points and leaves none of fifty runs at the target. Removing the memory features costs a further 4.0 points at one million steps, and the memory-free policy is still at 90.0% after 1.85M, having gained nothing from the extra 850k, so to that budget those features raise the ceiling and not the speed of reaching it. Neither decision is a refinement, and they do not fail alike: on the shared reward the policy collapses to 63.0% and loses to every baseline here, while without memory it holds 89.6%, ahead of Boids and the hybrid but short of both the target and APF. That second decision also qualifies the headline: the memory features are information Boids and APF do not have, so the comparison is between a learned controller with map memory and classical controllers without it, and on equal information APF wins at both budgets tested. A third result came from auditing the apparatus: repeating training at four seeds gives a 7.8-point across-seed spread at one million steps, which halves by 1.85M where every seed passes APF and the policy reaches 98.8% at 20% density.

Contributions. A four-way comparison on identical matched layouts with 50 replications, paired significance tests, effect sizes and bootstrap intervals, none of which is reported by any of the eight studies surveyed in Table I. A controlled measurement of the credit-assignment problem, in which the reward scheme is the only variable and costs more coverage than the gap between any two controllers here. Reporting of variance across training runs, which the surveyed literature omits. Evidence that percentage coverage over free cells misleads when free area varies. And a working decentralised MARL coverage system with the training configuration that makes it learnable identified explicitly.

Limitations. The simulation is a 2D kinematic grid with perfect sensing and localisation, no inertia, wind or battery model, so only the relative ordering plausibly transfers. The learned controller carries memory features the classical controllers lack, and the ablation shows these lift its ceiling, so the headline comparison is not one of equal information. The main experiment reports one training seed per condition at a budget the seed study shows to be undertrained, and that study covers 20% density only, so the density-dependent claims are not settled. Swarm size was fixed at six, hyperparameters were tuned only in the rollout-length dimension, the sensor-noise experiment is withdrawn, and a separate policy is trained per density rather than one generalising across them.

Future work. The immediate step is to repeat the four-way comparison with several training seeds per cell at a budget past convergence, which the 20% result suggests would turn parity into an advantage for the learned controller. Beyond that, one policy trained across all densities would test generalisation over environment statistics as well as layouts; minimal communication would test whether the 30% deficit is a partial-observability or a policy limit; and a valid sensor-noise experiment would give the robustness ordering the withdrawn one could not. Longer term, PyBullet dynamics and Crazyflie flights at the Bristol Robotics Laboratory would test whether the ordering survives hardware.

REFERENCES

- [1] M. Erdelj, M. Król, and E. Natalizio, "Wireless sensor networks and multi-UAV systems for natural disaster management," *Computer Networks*, vol. 124, pp. 72–86, 2017.
- [2] M. Dorigo, G. Theraulaz, and V. Trianni, "Swarm robotics: Past, present, and future," *Proceedings of the IEEE*, vol. 109, no. 7, pp. 1152–1165, 2021.
- [3] H. Zhu, Y. Wang, Z. Ma, and X. Li, "A comparative study of swarm intelligence algorithms for ucav path planning," *Applied Sciences*, vol. 13, no. 14, p. 8290, 2023.
- [4] C. W. Reynolds, "Flocks, herds and schools: A distributed behavioral model," in *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '87)*. ACM, 1987, pp. 25–34.
- [5] X.-p. Xu, X.-t. Yan, W.-y. Yang, K. An, W. Huang, and Y. Wang, "Algorithms and applications of intelligent swarm cooperative control: A comprehensive survey," *Progress in Aerospace Sciences*, vol. 135, p. 100869, 2022.
- [6] Y. Alqudsi and M. Makaraci, "Uav swarm coordination and control: A comprehensive review of architectures, algorithms and applications," *Journal of Field Robotics*, 2024.
- [7] K. Arshid, M. A. Khan, and S. Hussain, "Toward autonomous uav swarm navigation: A review of trajectory design paradigms," *Sensors*, vol. 25, no. 18, p. 5789, 2025.
- [8] Z. Shang, J. Qi, Y. Peng, M. Wang, C. Wu, and Y. Ping, "DEBS: Decentralized exploration planning for multi-UAV systems using bio-inspired self-aggregation mechanism," *Physical Communication*, vol. 73, p. 102898, 2025.
- [9] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, vol. 2, 1986, pp. 500–505.
- [10] H. Choset, "Coverage for robotics: A survey of recent results," *Annals of Mathematics and Artificial Intelligence*, vol. 31, pp. 113–126, 2001.
- [11] T. Hickling, M. Hogan, A. Tammam, and N. Aouf, "Deep reinforcement learning based autonomous decision-making for cooperative UAVs: A search and rescue real world application," *arXiv preprint arXiv:2502.20326*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.20326>
- [12] Y. Du, "Multi-UAV search and rescue with enhanced A* algorithm path planning in 3D environment," *International Journal of Aerospace Engineering*, vol. 2023, p. 8614117, 2023.
- [13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017. [Online]. Available: <https://arxiv.org/abs/1707.06347>
- [14] P. Fusco, L. Porcelli, F. Palmieri, and M. Ficco, "Supporting UAVs swarm missions by multi-agent reinforcement learning," in *2025 International Conference on Computing, Networking and Communications (ICNC), Workshop on Computing, Networking and Communications (CNC)*. IEEE, 2025, pp. 222–226.
- [15] J. Yu, H. Zhang, and Y. Zhang, "Distributed UAV swarms for 3D urban area coverage with incomplete information using event-triggered hierarchical reinforcement learning," *Computers & Industrial Engineering*, vol. 199, p. 110734, 2025.
- [16] W. L. Leong, J. Cao, and R. S. H. Teo, "Dynamic decentralized 3D urban coverage and patrol with UAVs," in *arXiv preprint arXiv:2406.09828*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.09828>
- [17] Z. Xie, Q. Wang, B. Kong, and S. Gao, "Enhanced coverage path planning strategies for UAV swarms based on SADQN algorithm," *Computers, Materials & Continua*, 2025.
- [18] J. N. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, 2018, pp. 2974–2982.
- [19] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls, and T. Graepel, "Value-decomposition networks for cooperative multi-agent learning based on team reward," in *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, 2018, pp. 2085–2087.
- [20] T. Rashid, M. Samvelyan, C. S. de Witt, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 4295–4304.
- [21] J. Kazil, D. Masad, and A. Crooks, "Utilizing python for agent-based modeling: The mesa framework," *Social, Cultural, and Behavioral Modeling (SBP-BRIMS)*, pp. 308–317, 2020.
- [22] D. Perkins, "Probability of detection for the search manager," The Centre for Search Research, Ashington, Northumberland, UK, Tech. Rep., 2013. [Online]. Available: <https://csr.org.uk/media/fsldirix/2013-probability-of-detection-for-the-search-manager.pdf>

- [23] National Search and Rescue Committee, "United states national search and rescue supplement to the international aeronautical and maritime search and rescue manual," United States Coast Guard, Washington, DC, Tech. Rep., 2018.
- [24] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: <http://jmlr.org/papers/v22/20-1364.html>
- [25] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *The Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947.
- [26] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [27] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions," *Psychological Bulletin*, vol. 114, no. 3, pp. 494–509, 1993.
- [28] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.